

Design & Performance Analysis of Pipelined MIPS Processor with Branch Prediction

Course Project Presentation



Advaith Kiran M - 23M1156

EE739 – Processor Design

Department of Electrical Engineering

Indian Institute of Technology Bombay

Project Overview

Agenda

- To design and verify a 5-stage pipelined MIPS processor with hazard handling and branch prediction
- Quantitative Performance evaluation with and without branch prediction

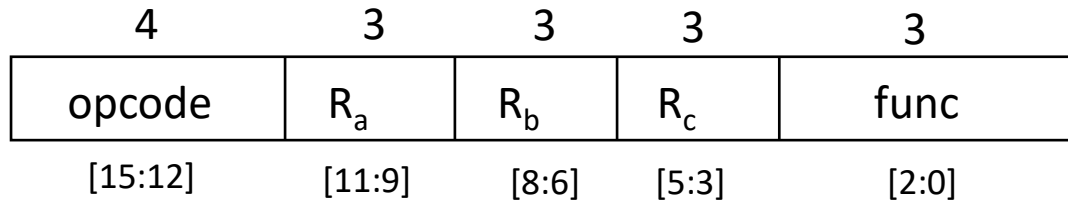
Design Goals

- 5-stage pipelined datapath : **IF → ID → EX → MEM → WB**
- Data hazard resolution using forwarding and stalling
- Control hazard recovery using pipeline flush
- Bimodal branch predictor to reduce control hazard penalty
- Program-driven performance evaluation with and without branch prediction

Supported ISA

Instruction Type	Instruction Format	Instruction/Function															
R-type	<table><tr><td>4</td><td>3</td><td>3</td><td>3</td><td>3</td></tr><tr><td>opcode</td><td>R_a</td><td>R_b</td><td>R_c</td><td>func</td></tr><tr><td>[15:12]</td><td>[11:9]</td><td>[8:6]</td><td>[5:3]</td><td>[2:0]</td></tr></table>	4	3	3	3	3	opcode	R _a	R _b	R _c	func	[15:12]	[11:9]	[8:6]	[5:3]	[2:0]	ADD, SUB, AND, SLT, SRL
4	3	3	3	3													
opcode	R _a	R _b	R _c	func													
[15:12]	[11:9]	[8:6]	[5:3]	[2:0]													
I-type	<table><tr><td>4</td><td>3</td><td>3</td><td>6</td></tr><tr><td>opcode</td><td>R_a</td><td>R_b</td><td>Imm6</td></tr><tr><td>[15:12]</td><td>[11:9]</td><td>[8:6]</td><td>[5:0]</td></tr></table>	4	3	3	6	opcode	R _a	R _b	Imm6	[15:12]	[11:9]	[8:6]	[5:0]	ADDI, LOAD, STORE, BEQ			
4	3	3	6														
opcode	R _a	R _b	Imm6														
[15:12]	[11:9]	[8:6]	[5:0]														
J-type	<table><tr><td>4</td><td>12</td></tr><tr><td>opcode</td><td>Imm12</td></tr><tr><td>[15:12]</td><td>[11:9]</td></tr></table>	4	12	opcode	Imm12	[15:12]	[11:9]	JUMP									
4	12																
opcode	Imm12																
[15:12]	[11:9]																

R-type instruction



Execution semantics:

$RF[R_c] \leftarrow RF[R_a] \text{ op } RF[R_b]$

Instruction:

$I_0 = \{ R_type, R1, R2, R3, f_ADD \}$

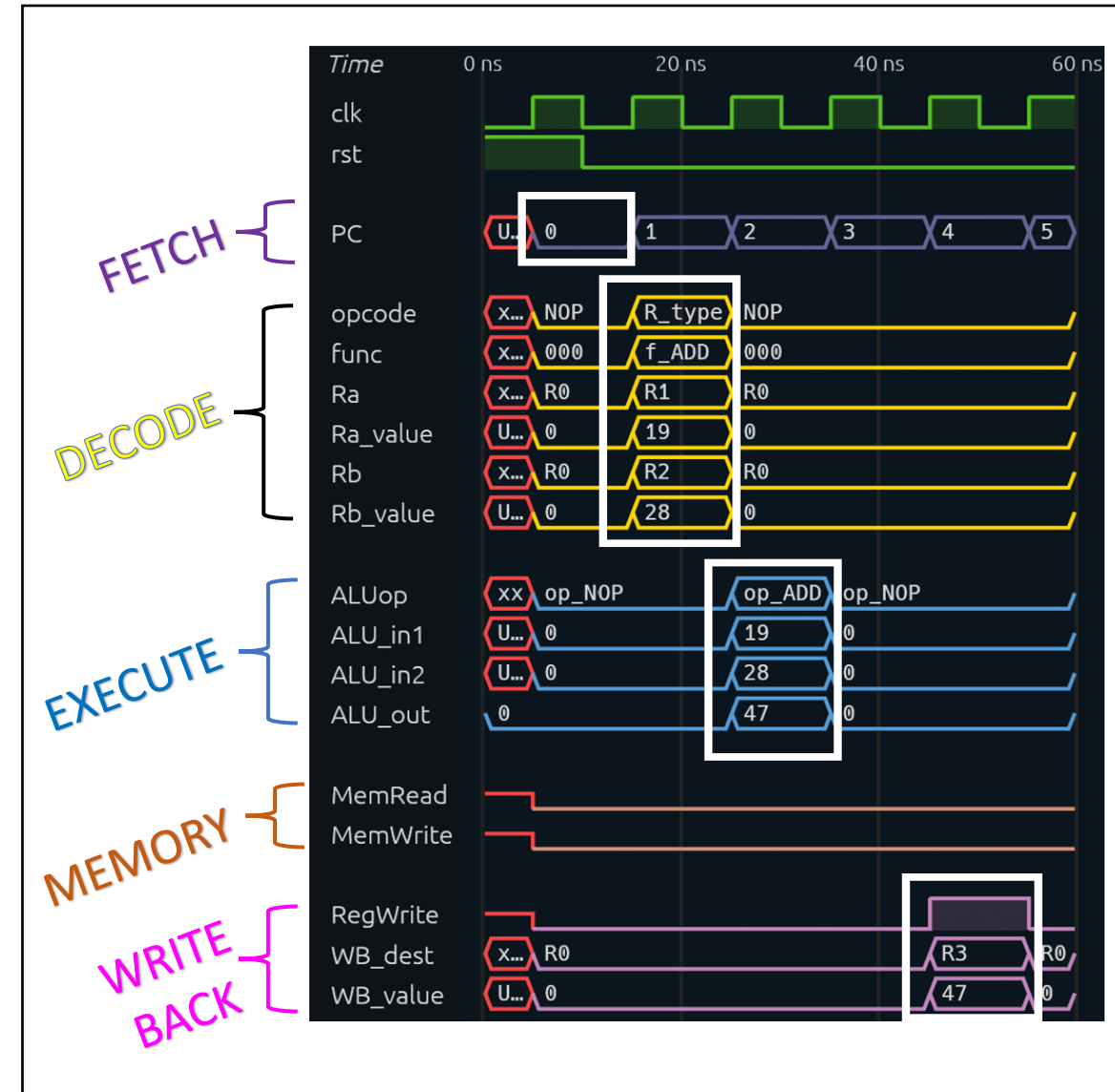
Initial Values:

$RF[R1] = 19$

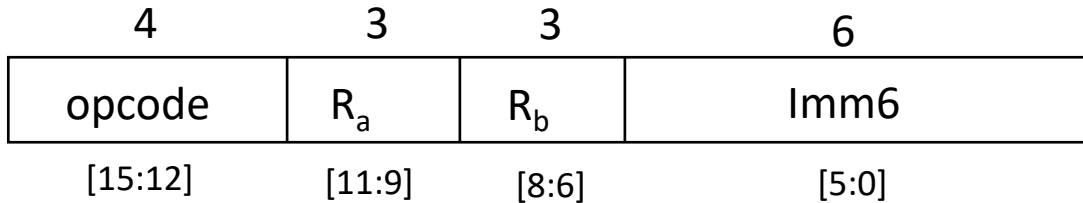
$RF[R2] = 28$

Expected Result:

$RF[R3] = 47$



I-type ALU instruction



Execution semantics:

$RF[R_b] \leftarrow RF[R_a] \text{ op Imm6_ext}$

Instruction:

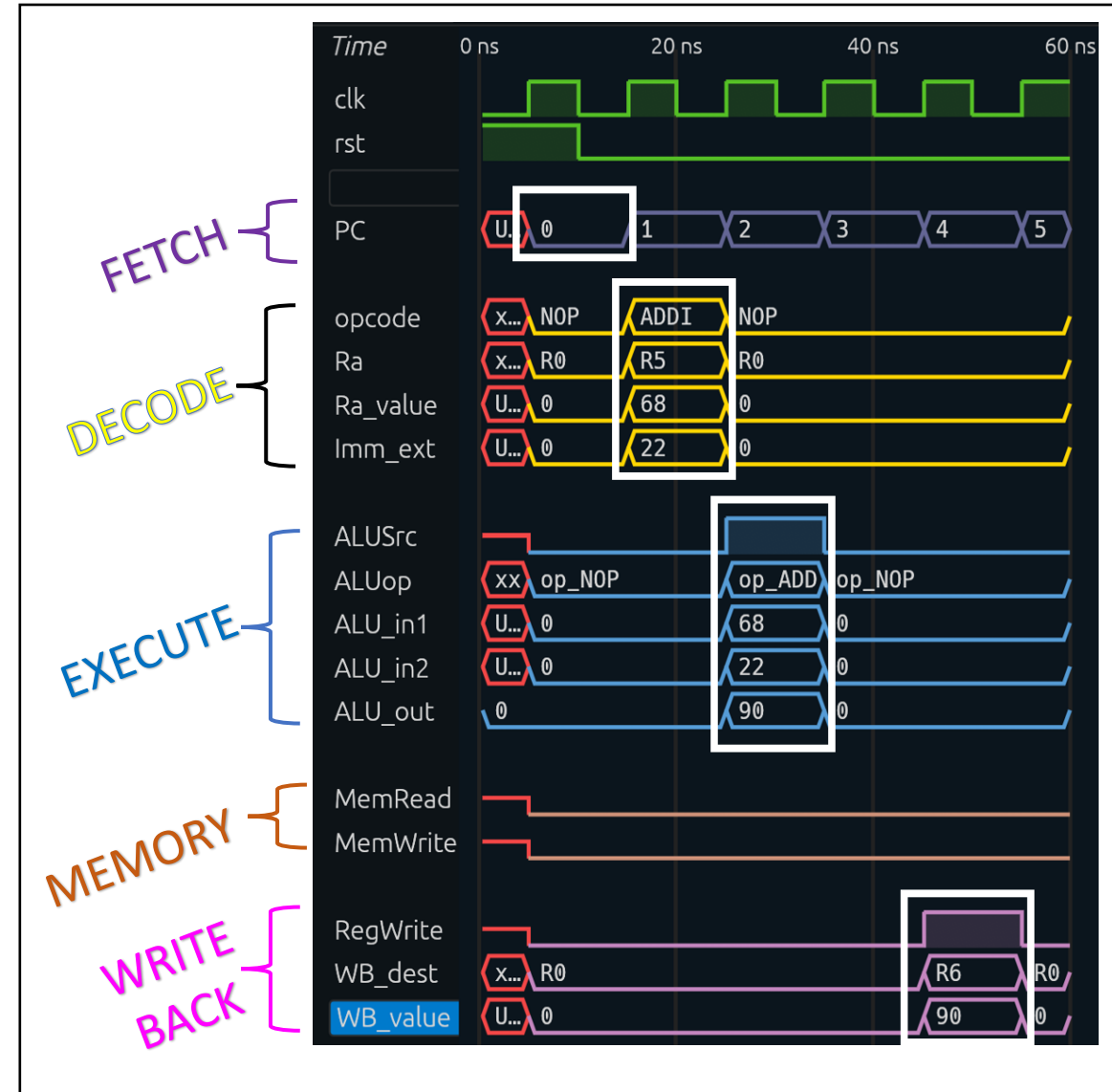
$I_0 = \{ \text{ADDI}, R5, R6, 6'd22 \}$

Initial Values:

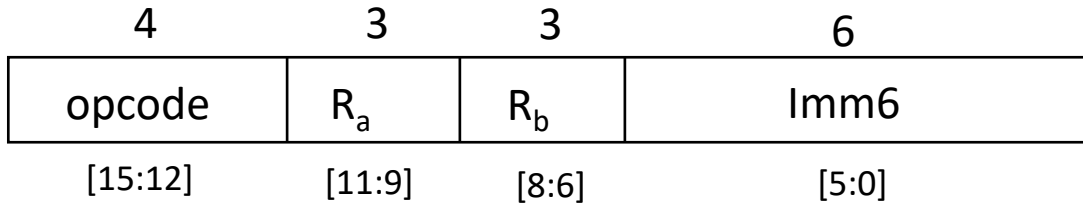
$RF[R5] = 68$

Expected Result:

$RF[R6] = 90$



LOAD instruction



Execution semantics:

$RF[R_b] \leftarrow Dmemory \{ RF[R_a] + Imm6_ext \}$

Instruction:

$I_0 = \{ \text{LOAD}, R1, R7, 6'd31 \}$

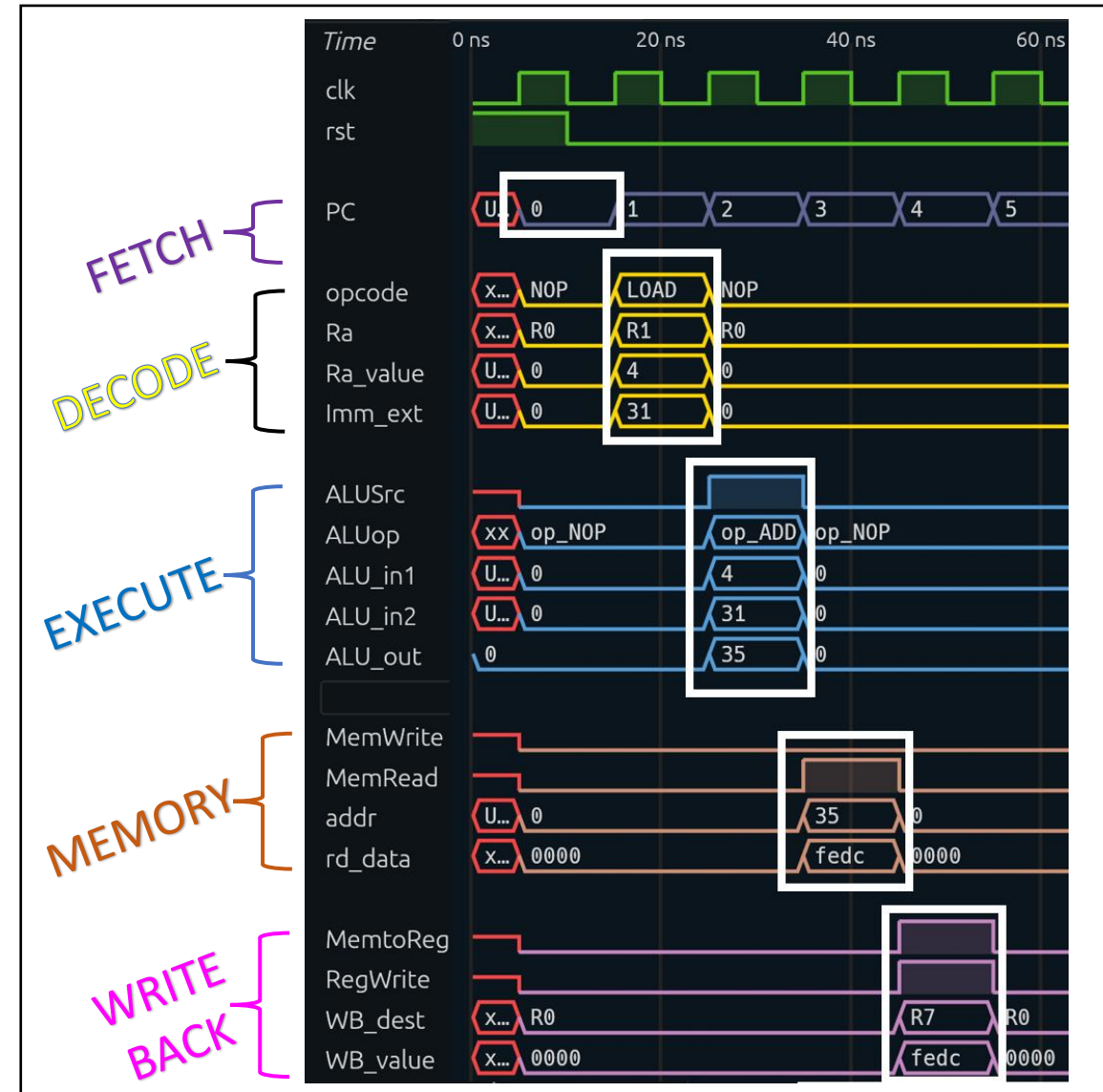
Initial Values:

$RF[R1] = 4$

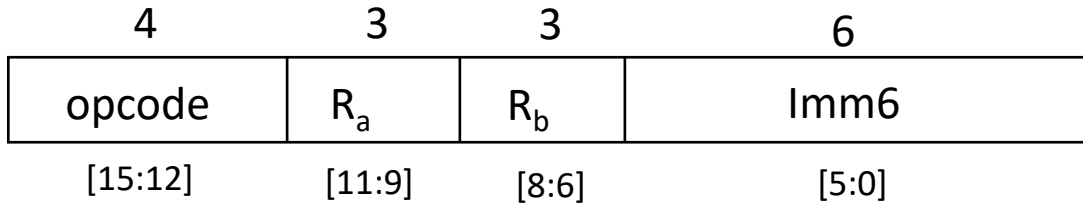
$DM[35] = 16'hFEDC$

Expected Result:

$RF[R7] = 16'hFEDC$



STORE instruction



Execution semantics:

$\text{Dmemory} \{ \text{RF}[R_a] + \text{Imm6_ext} \} \leftarrow \text{RF}[R_b]$

Instruction:

$I_0 = \{ \text{STORE}, R2, R7, 6'd23 \}$

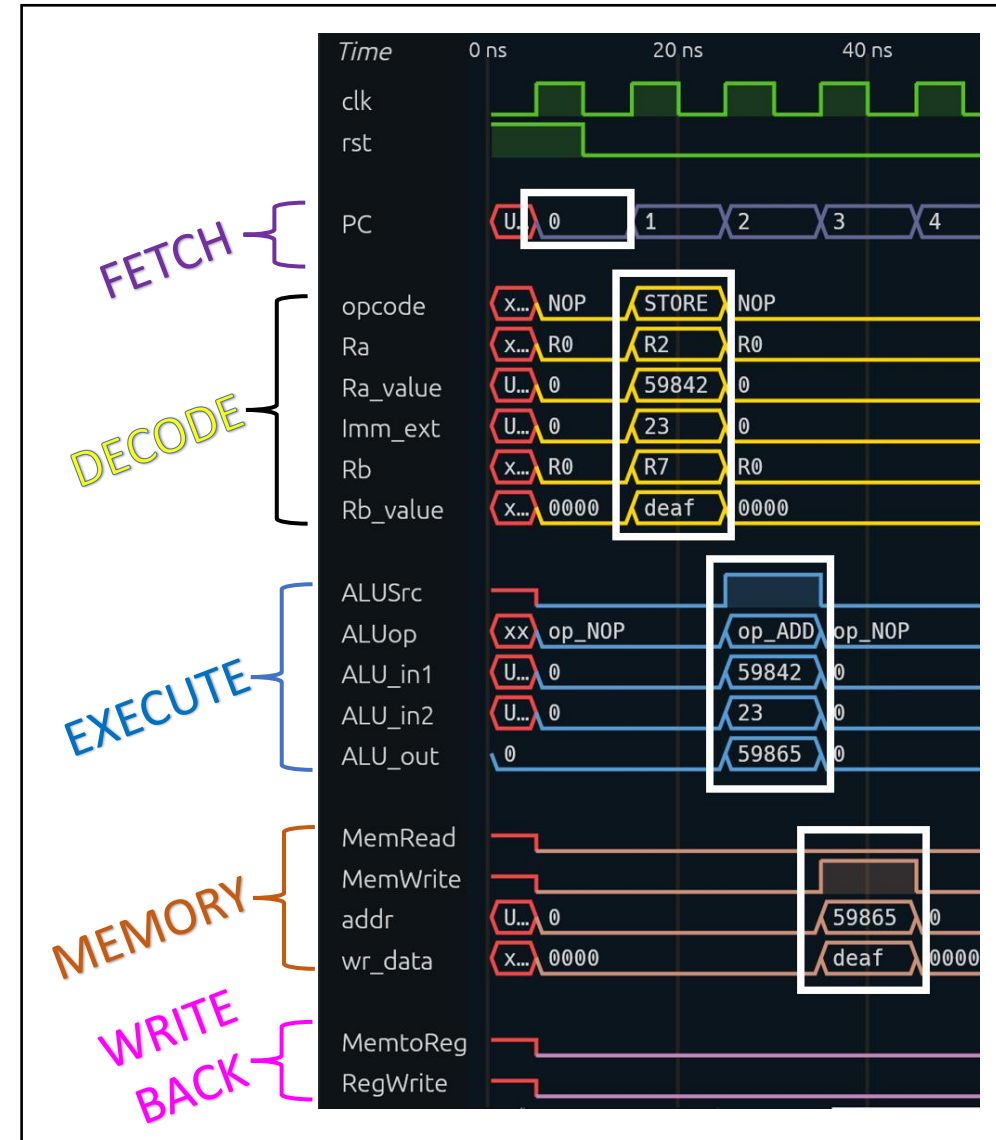
Initial Values:

$\text{RF}[R2] = 16'd59842$

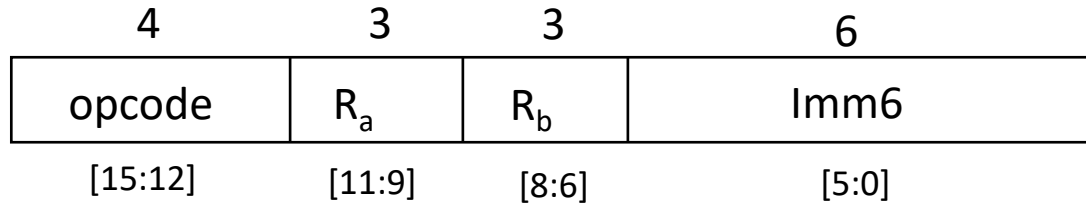
$\text{RF}[R7] = 16'hDEAF$

Expected Result:

$\text{DM}[25] = 16'hDEAF$



BEQ instruction



Execution semantics:

$PC \leftarrow RF[R_a] == RF[R_b]$
? (PC_plus1 + Imm6_ext)
: PC_plus1

Instruction:

$I_0 = \{ \text{BEQ}, R3, R4, 6'd23 \}$

Initial Values:

RF[R3] = AAAA

RF[R4] = AAAA

Expected Result:

Branch_taken = 1

Target_PC = 24

PC redirects to target address

Initial Values:

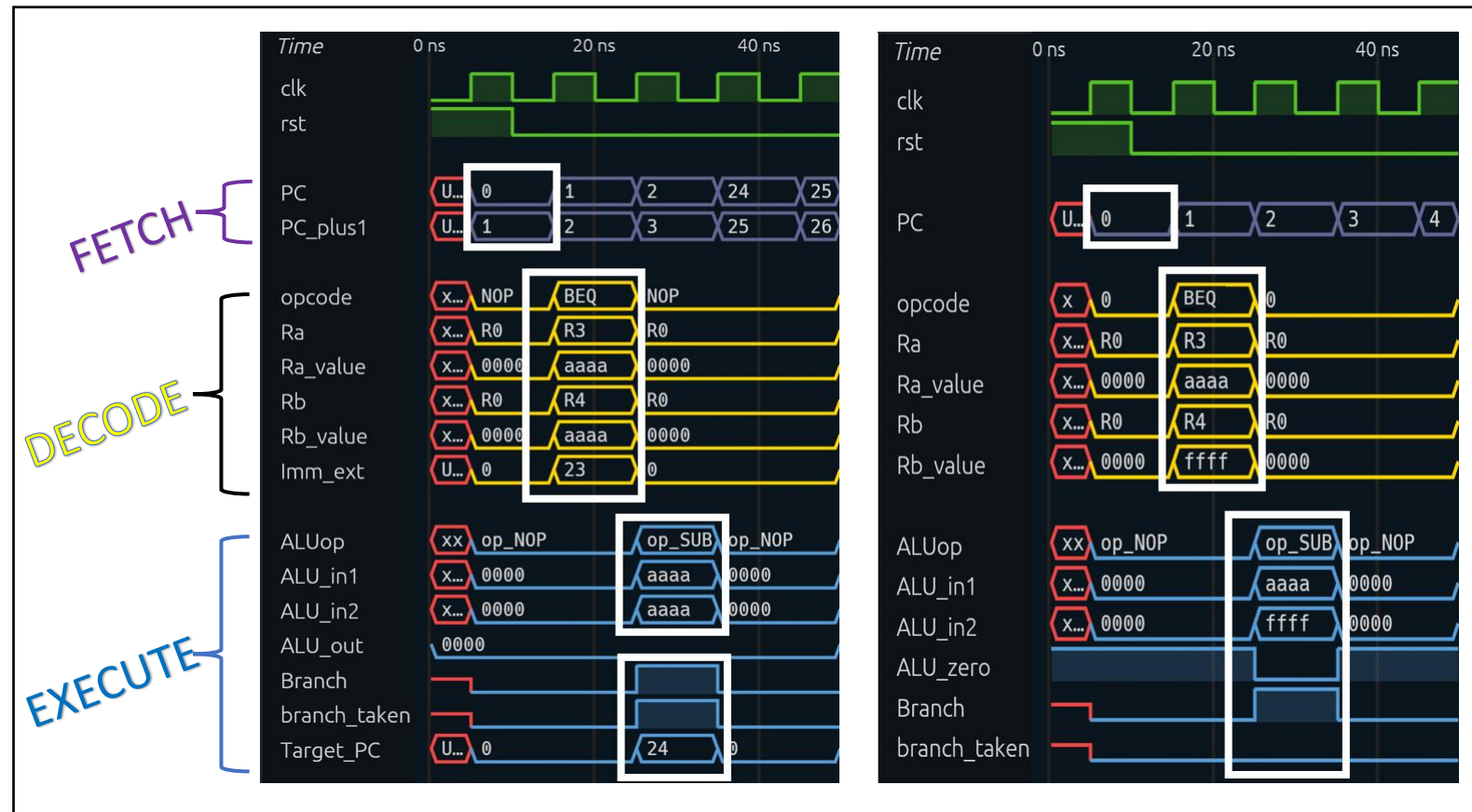
RF[R3] = AAAA

RF[R4] = FFFF

Expected Result:

Branch_taken = 0

PC doesn't redirect to target address



JUMP instruction



Execution semantics:

$PC \leftarrow \{ PC_plus1 [15:12], Imm12 \}$

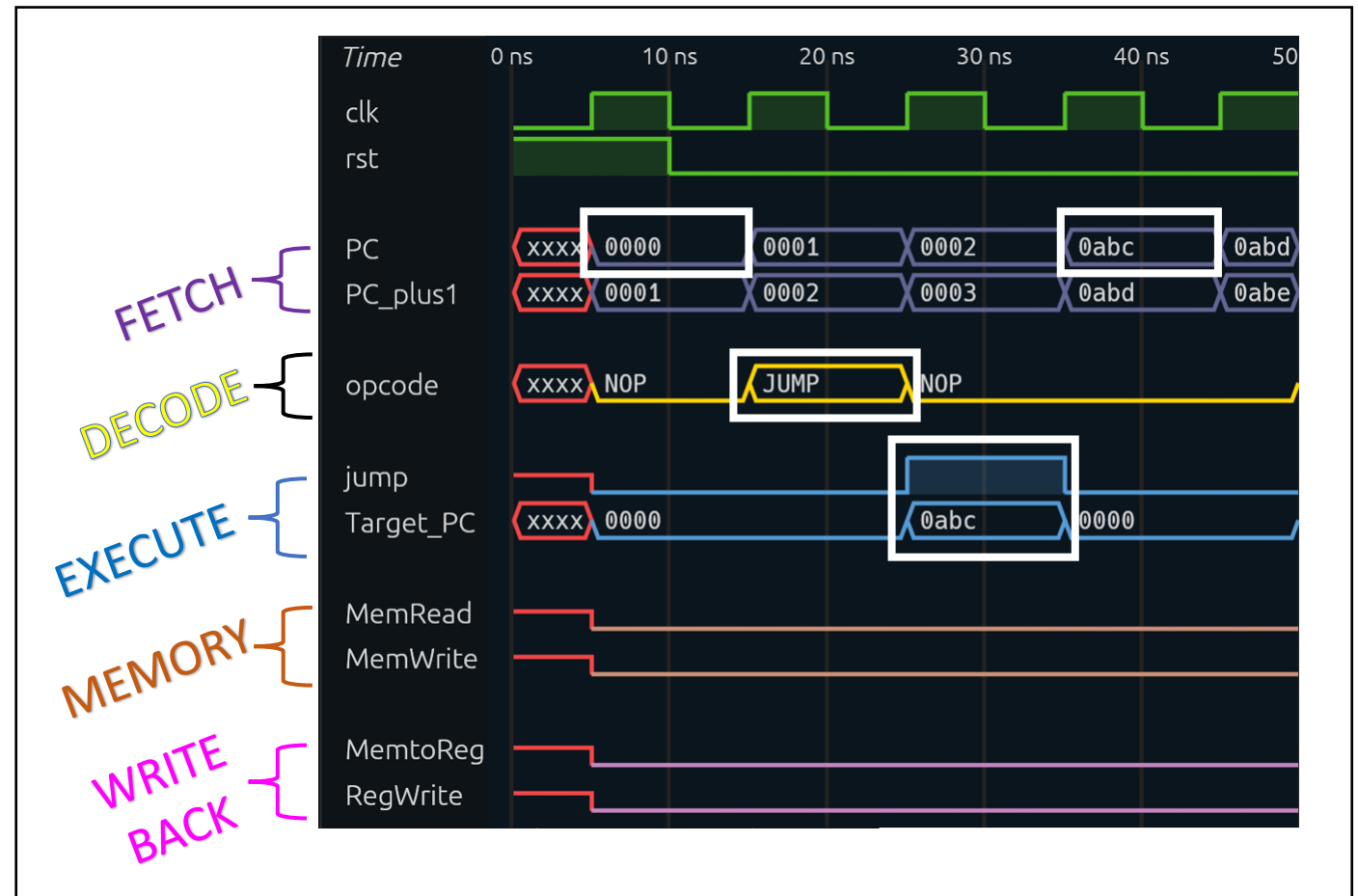
Instruction:

$I_0 = \{ JUMP, 12'hABC \}$

Expected Result:

Target_PC = 16'h0ABC

PC redirects to target address



Forwarding Unit - resolving RAW hazards

- I_2 depends on results from I_0 and I_1 .
- Forwarding provides the latest values of R3 and R6 without stalling.

Instruction sequence:

$I_0 = \{ R_type, R1, R2, R3, f_ADD \}$

$I_1 = \{ R_type, R4, R5, R6, f_ADD \}$

$I_2 = \{ R_type, R3, R6, R7, f_ADD \}$

Initial Values:

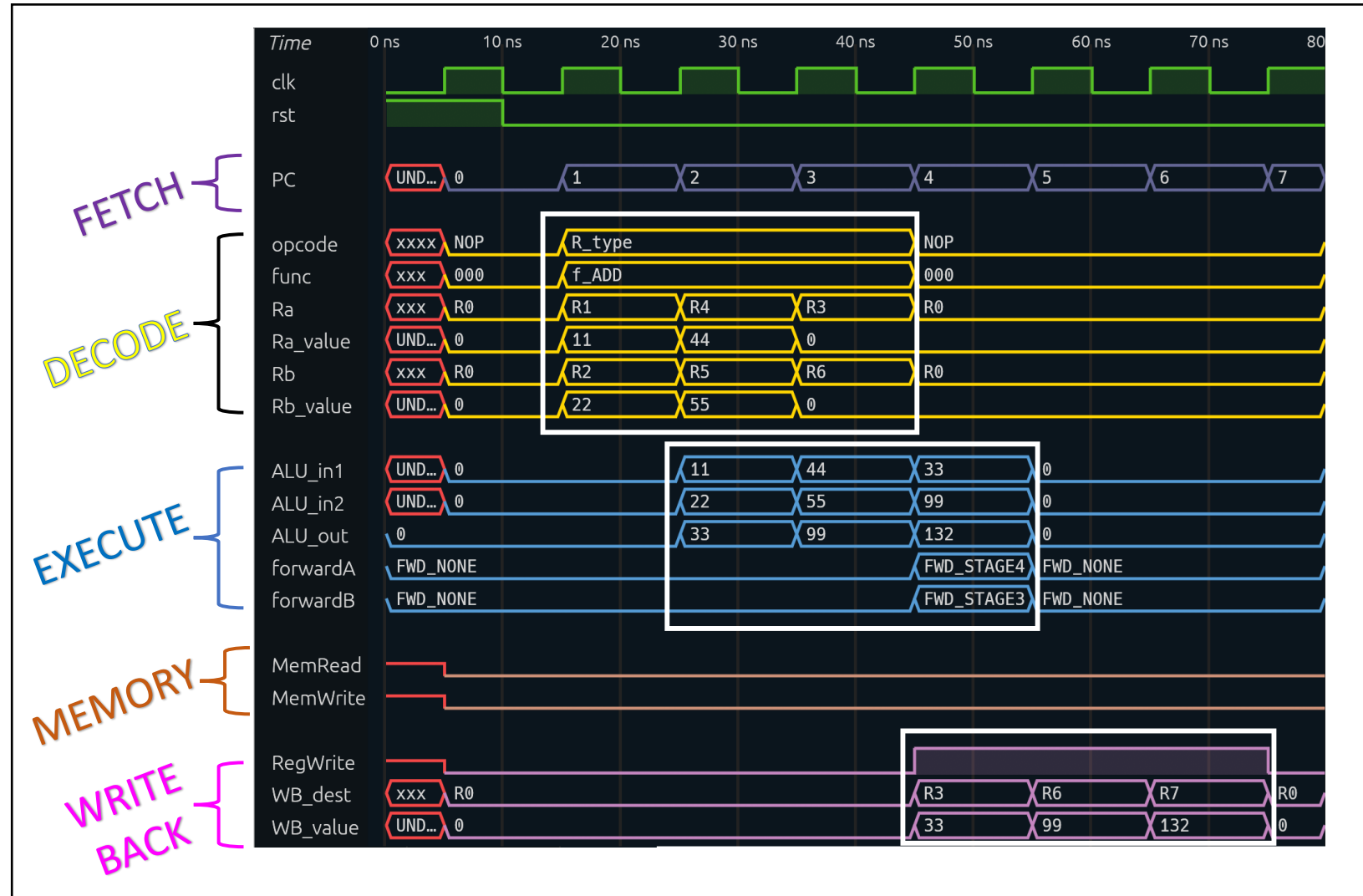
$RF[R1] = 11, RF[R2] = 22$

$RF[R4] = 44, RF[R5] = 55$

$RF[R3] = 0$

Expected Result:

$RF[R7] = 33 + 99 = 132$



Forwarding Priority – selecting the most recent result

- I_2 reads R3, which is written by both I_0 and I_1 .
- The forwarding unit selects the latest R3 value from I_1 .

Instruction sequence:

$I_0 = \{ R_type, R1, R2, R3, f_ADD \}$

$I_1 = \{ R_type, R3, R4, R3, f_ADD \}$

$I_2 = \{ R_type, R5, R3, R6, f_ADD \}$

Initial Values:

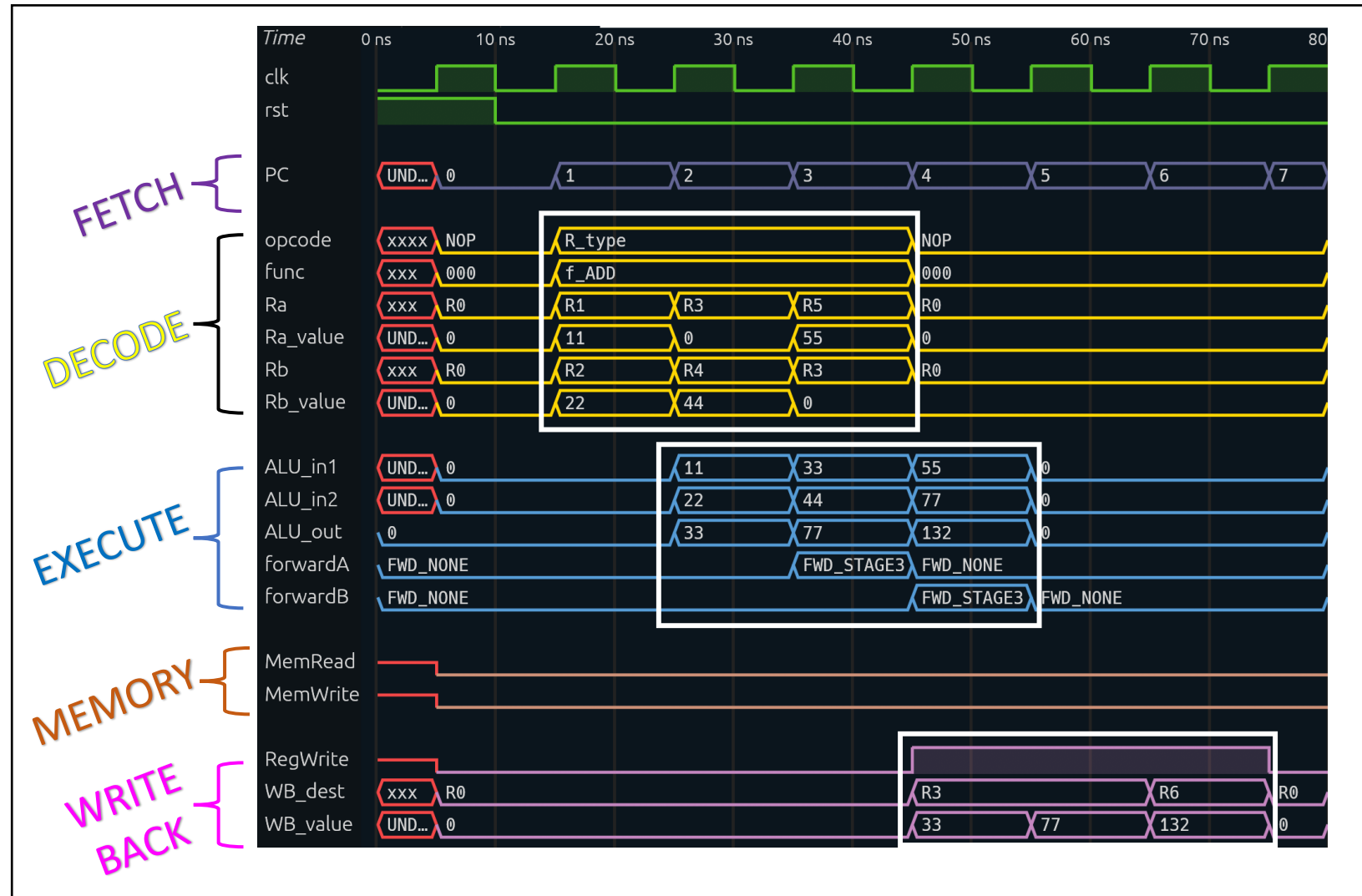
$RF[R1] = 11, RF[R2] = 22$

$RF[R4] = 44, RF[R5] = 55$

$RF[R3] = 0$

Expected Result:

$RF[R6] = 55 + 77 = 132$



Hazard Detection Unit – load-use stall control

- I_1 immediately uses the value loaded into R2.
- The hazard detection unit inserts one stall cycle before I_1 enters EX.

Instruction sequence:

$I_0 = \{ \text{LOAD}, R1, R2, 6'd4 \}$

$I_1 = \{ R_type, R2, R3, R5, f_ADD \}$

Initial Values:

$RF[R1] = 11$

$DM[15] = 22$

$RF[R3] = 33$

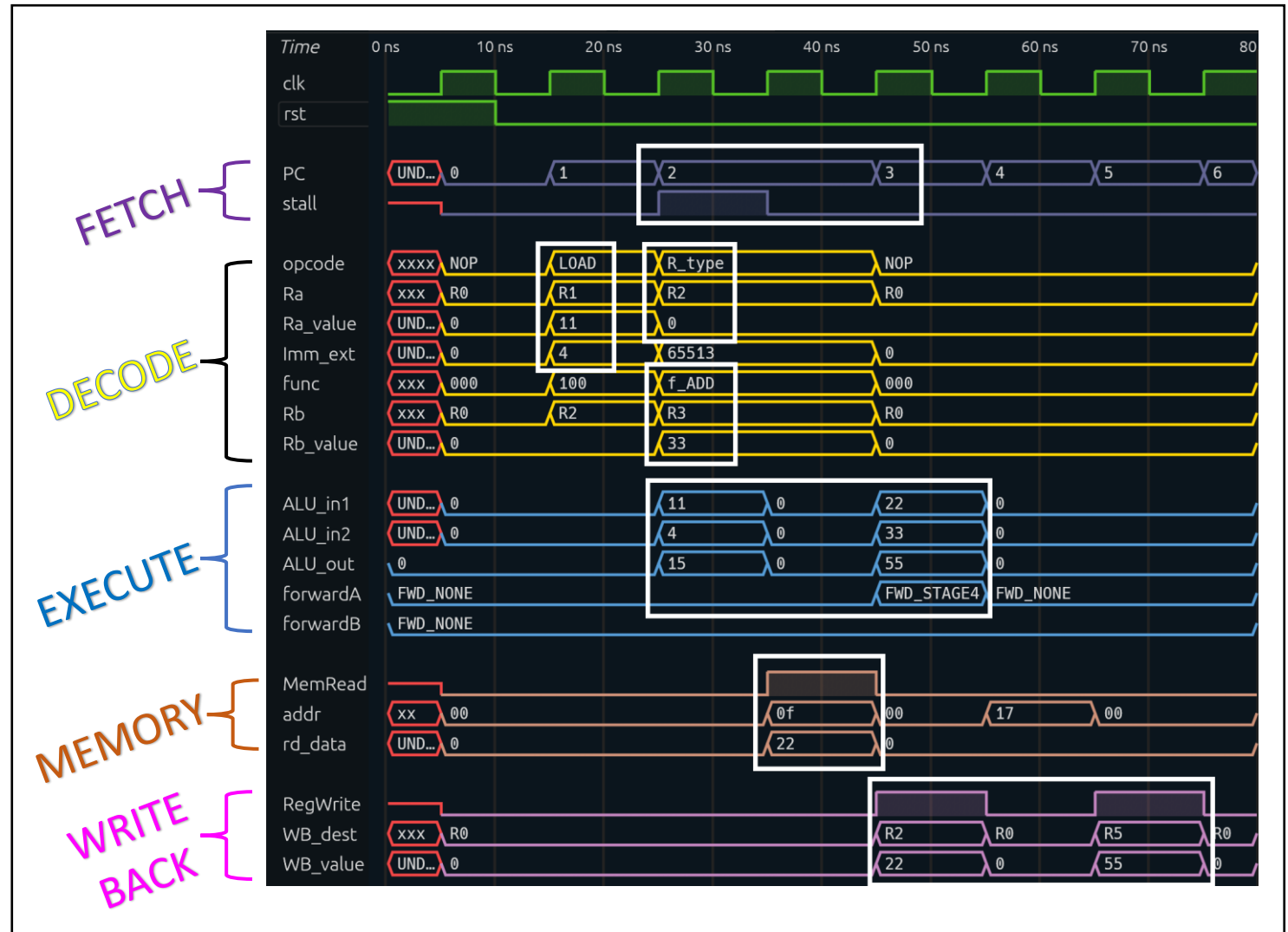
Expected Result:

stall = 1

One bubble inserted

$RF[R2] = 22$

$RF[R5] = 55$



BEQ Control Hazard – conditional PC redirect & pipeline flush

- Since the BEQ condition is true; branch_taken and flush go high in the Execute stage.
- PC redirects to the branch target address, and the wrong-path instructions are flushed without updating registers or memory.

Instruction sequence:

$I_0 = \{ \text{BEQ}, R1, R2, 6'd9 \}$

$I_1 = \{ R_type, R3, R4, R5, f_ADD \}$

$I_2 = \{ \text{ADDI}, R1, R4, 6'd31 \}$

Initial Values:

RF [R1] = 11, RF [R2] = 11

RF [R3] = 33, RF [R4] = 44

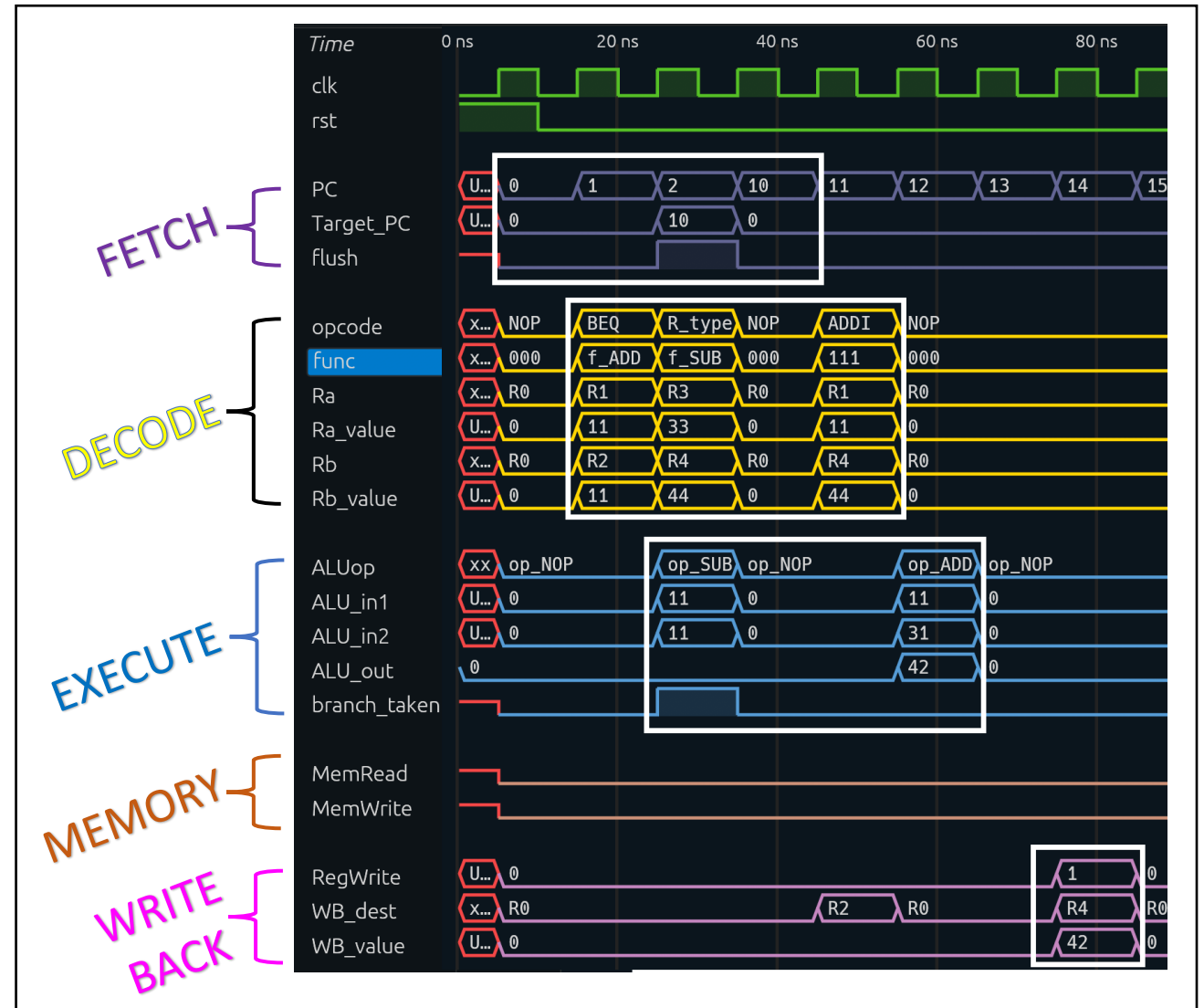
Expected Result:

branch_taken = 1

flush = 1

PC redirects to Target_PC

Wrong-path instructions are killed



JUMP Control Hazard – unconditional PC redirect & pipeline flush

- JUMP is resolved in the Execute stage; jump and flush go high, and Target_PC is selected.
- Sequential wrong-path instructions are flushed, so they do not update registers or memory.

Instruction sequence:

$I_0 = \{ \text{JUMP}, 12'hABC \}$

$I_1 = \{ \text{R_type}, R1, R2, R3, f_ADD \}$

$I_2 = \{ \text{R_type}, R4, R5, R6, f_AND \}$

$I_{0ABC} = \{ \text{ADDI}, R1, R4, 6'd31 \}$

Initial Values:

$RF[R1] = 11, RF[R2] = 22$

$RF[R4] = 44, RF[R5] = 55$

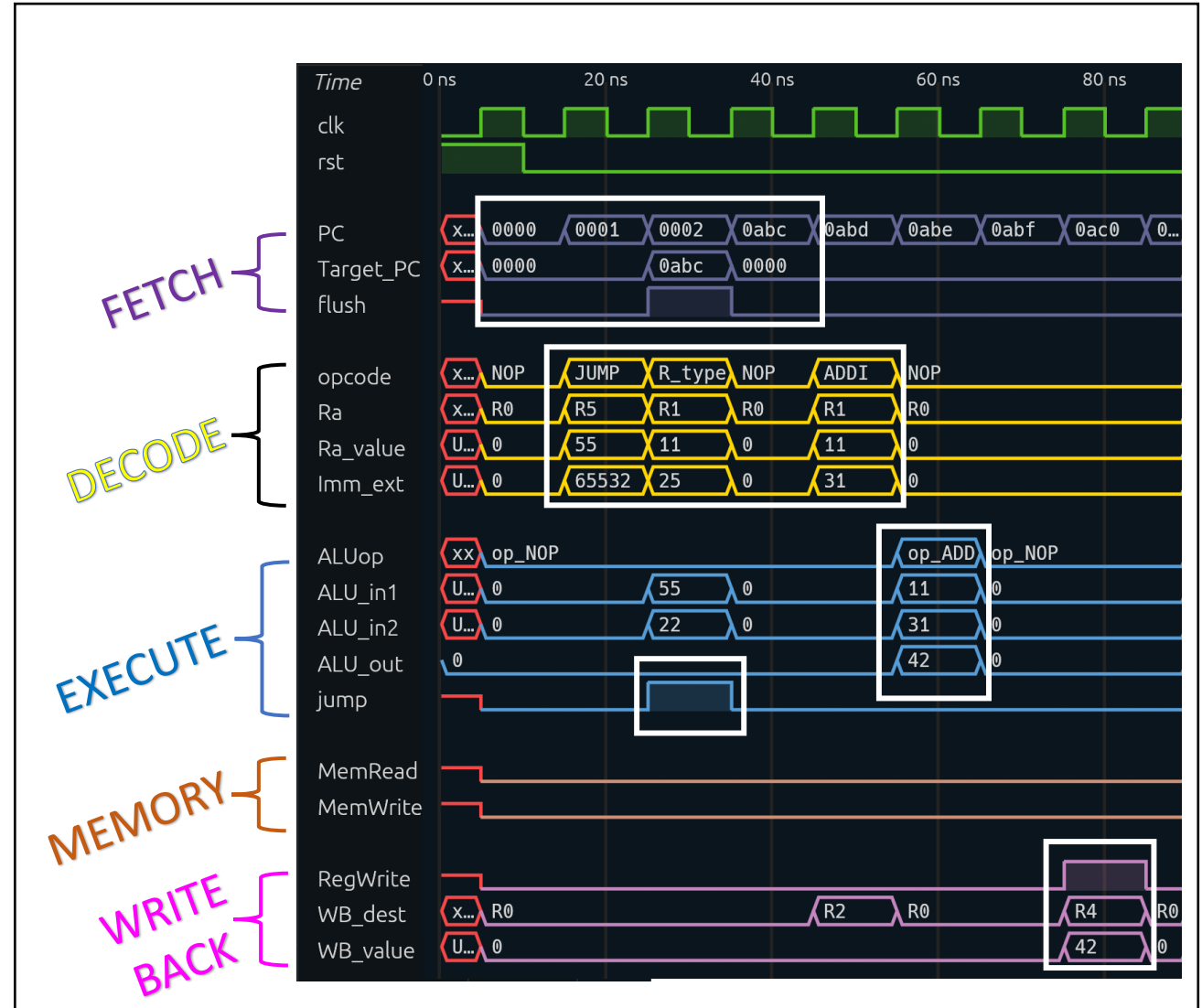
Expected Result:

jump = 1

flush = 1

PC redirects to 16'h0ABC

Wrong-path instructions are killed



Bimodal Branch Predictor – BEQ prediction

- On the first BEQ execution, the predictor selects the sequential PC, but the actual branch target is PC = 0; mispredict and flush go high.
- After update, the predictor stores the branch target and predicts the taken branch correctly in later loop iterations.

Instruction sequence:

$I_0 = \{ \text{ADDI}, R1, R1, 6'd1 \}$
 $I_1 = \{ R_type, R1, R2, R4, f_ADD \}$
 $I_2 = \{ \text{BEQ}, R6, R7, 6'b111101 \}$
 $I_3 = \{ \text{LOAD}, R0, R7, 6'd0 \}$
 $I_4 = \{ \text{STORE}, R0, R7, 6'd0 \}$

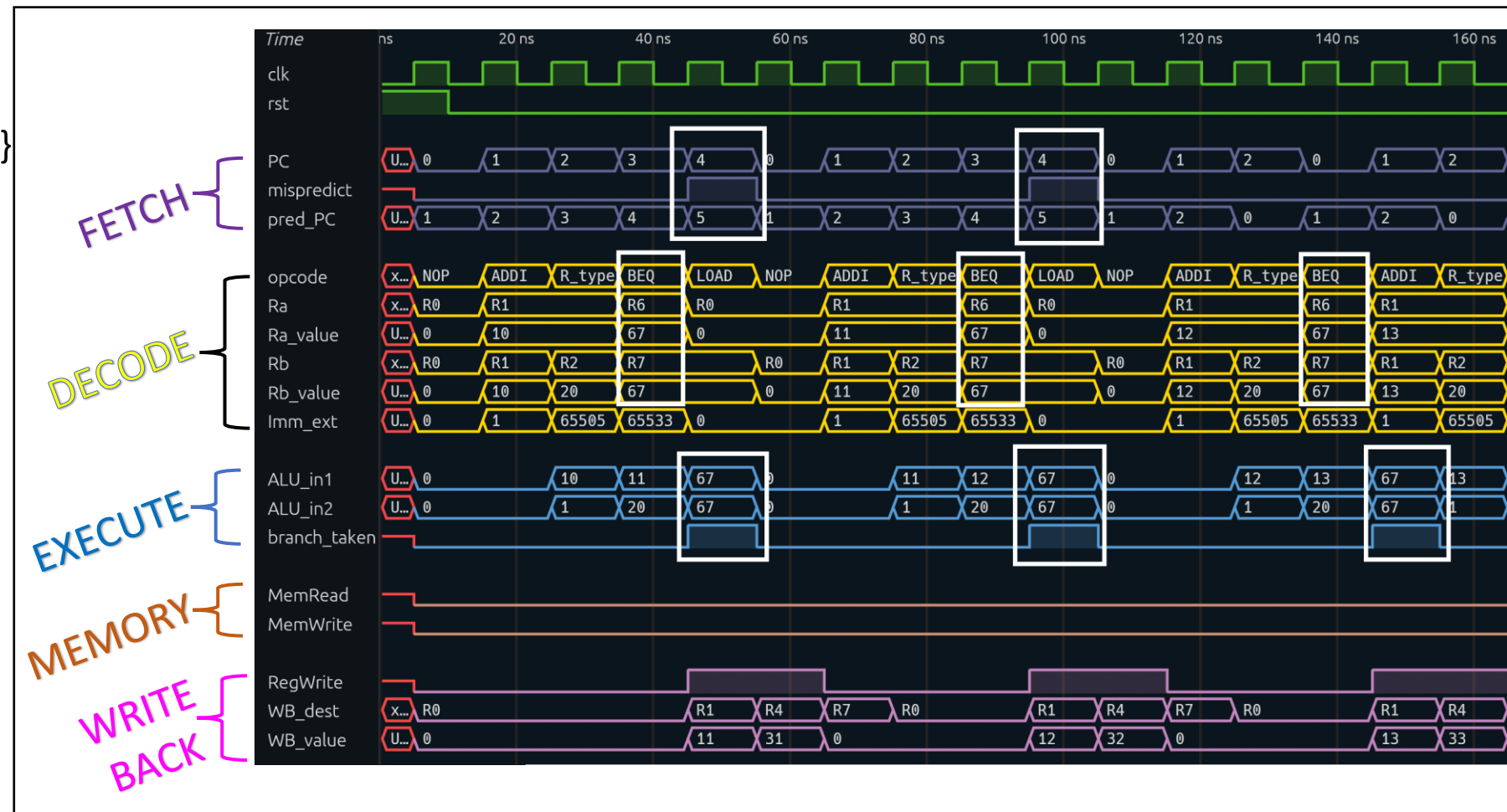
Initial Values:

RF[R6] = 67

RF[R7] = 67

Expected Result:

- BEQ condition is true
- First prediction misses
- Predictor updates target PC
- Later predictions are correct



Bimodal Branch Predictor – JUMP prediction

- On the first JUMP execution, the predictor has no stored target, so flush goes high and PC redirects to 0.
- After predictor update, the JUMP target is predicted correctly and the loop continues with reduced control hazard penalty.

Instruction sequence:

$I_0 = \{ \text{ADDI}, R1, R1, 6'd1 \}$

$I_1 = \{ R_type, R1, R2, R4, f_ADD \}$

$I_2 = \{ \text{JUMP}, 12'd0 \}$

$I_3 = \{ \text{LOAD}, R0, R7, 6'd0 \}$

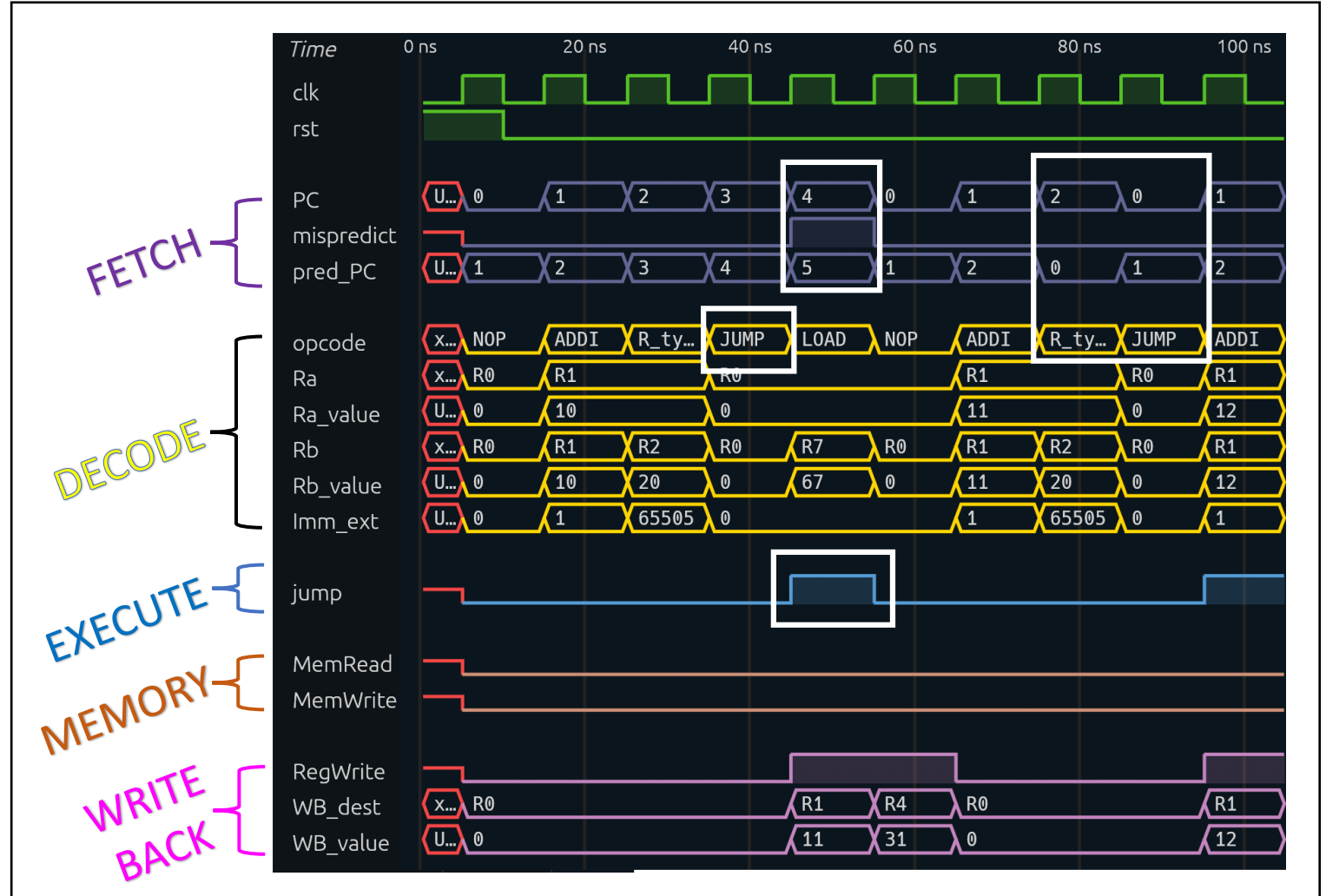
$I_4 = \{ \text{STORE}, R0, R7, 6'd0 \}$

Initial Values:

$RF[R1] = 10, RF[R2] = 20$

Expected Result:

- First jump prediction misses
- Predictor stores target PC = 0
- Later jumps are predicted correctly
- Wrong-path LOAD/STORE are flushed



Impact of branch prediction on processor performance

Algorithm	Test Condition	Cycle Count		Speedup	Performance Improvement
		without BP	with BP		
Linear search	Worst-case: key at last index	200	160	1.25x	25 %
Fibonacci sequence	Sequential term generation	245	175	1.4x	40 %
Bubble sort	Worst-case: reverse-sorted array	6845	5360	1.277x	27.7 %

**A fixed input size of 25 terms/elements was used across all benchmark programs for consistent performance comparison.*

$$\text{Speedup} = \frac{\text{cycle count without BP}}{\text{cycle count with BP}}$$

$$\text{Performance improvement} = (\text{Speedup} - 1) * 100\%$$

Average performance improvement = 30.9%

Conclusion

- 5-stage pipelined MIPS processor successfully implemented and verified.
- Forwarding, stalling, and flushing mechanisms improved pipeline correctness.
- Bimodal branch predictor reduced control hazard penalties.
- Benchmark simulations showed performance improvement with branch prediction.
- Average performance improvement achieved: **30.9%**.
- Loop-heavy programs benefited the most from branch prediction.

References

- [1] D. A. Patterson and J. L. Hennessy,
Computer Organization and Design RISC-V Edition:
The Hardware/Software Interface,
2nd ed., Morgan Kaufmann, 2020.
- [2] S. L. Harris and D. Harris,
Digital Design and Computer Architecture: RISC-V Edition,
1st ed., Morgan Kaufmann, 2021.